

Read the paper. Keep the trail.

An explanation and interview guide for Gagan P's AI/ML internship submission.

A 30-second description to practise

PaperTrail turns a research topic or arXiv ID into a briefing for one selected paper, then answers follow-up questions from retrieved source passages. Each accepted factual claim carries quotations and page references. The local application saves its state so a failed run can resume, and exports an evidence reader that reviewers can inspect.

What a user actually gets

INPUT	PROCESS	OUTPUT
A topic, arXiv ID or arXiv URL	Find and select one paper; fetch, parse and index it	Summary, problem, method, results and reported limitations
A question about a saved paper	Retrieve relevant passages; generate and check claims	A cited answer or an explicit insufficient-evidence response
An existing session ID	Read or export its durable saved state	Markdown, JSON and a portable HTML evidence reader

Three ideas to keep distinct

- **RAG:** retrieve text relevant to a question before asking a model to answer. This changes the evidence supplied at inference time; it does not train the model.
- **State graph:** a finite set of named steps and allowed transitions. Python owns the workflow; the model returns structured content within that workflow.
- **Provenance:** a route from a claim to the exact source passage. It helps a person audit an answer, but it does not prove that the interpretation is correct.

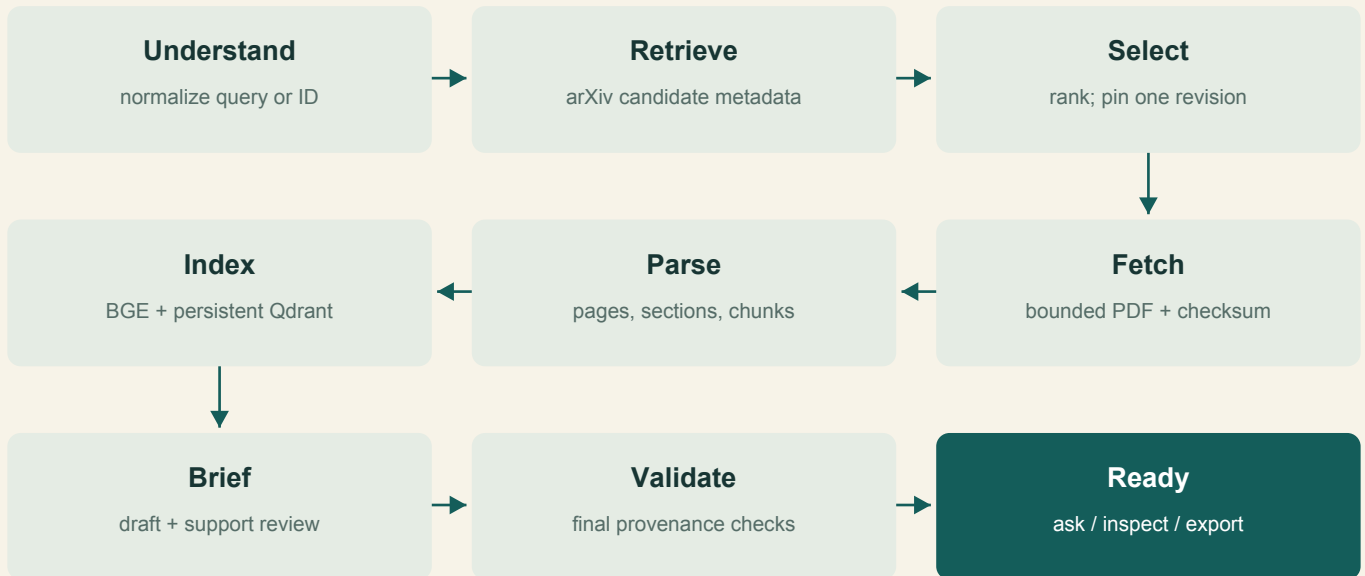
Scope you can describe precisely

One paper per session. Qwen3.5 4B through Ollama, CPU embeddings, Qdrant local storage and SQLite checkpoints. The CLI and local browser UI share one agent. HTML exports and the public Pages demo display saved results.

A workflow you can inspect

The graph makes progress and failure explicit. Each successful node advances the saved checkpoint.

MAIN PIPELINE - EXECUTABLE TRANSITIONS IN GRAPH.PY



QA loop: Ready -> retrieve passages -> generate and validate answer -> save exchange -> Ready.

STATE OR ARTIFACT	WHY IT EXISTS
SQLite RunState	Next node, selected paper, model names, errors, event timings, briefing and conversation history
Original PDF + parsed.json	Source checksum, page/section text, stable passage IDs and offsets
Qdrant local collection	Reusable passage vectors and payloads tied to the PDF, embedding model and parser version

What happens when generation fails?

The saved node remains the failed step. After fixing the cause, resume re-enters that step. Completed earlier nodes stay completed. Stable vector IDs make repeated index upserts idempotent. This is retry-safe progress, not a promise that external work happens exactly once.

From relevant text to a claim

Retrieval decides what the model can see. Validation decides which drafts the application will accept.

1 / Retrieve complementary evidence

Dense retrieval uses BGE-small-en-v1.5 through FastEmbed on CPU to find related meaning. **BM25** rewards exact terms, which helps with method names and technical vocabulary. Reciprocal-rank fusion combines ranks using **1 / (60 + rank)** from each list; similarity and keyword scores are not comparable probabilities.

Chunks stay within one page and retain section labels. Briefing retrieval seeks problem, method, results and limitations. QA selects diverse passages and suppresses heavy overlap. References are excluded unless requested. A follow-up can reuse the previous question for retrieval; earlier generated answers are never source evidence.

2 / Let code own the quotations

STEP	WHAT IT CHECKS OR CONTROLS
Evidence aliases	Python splits retrieved text into sentence-like spans and assigns short IDs such as E1. The model selects IDs and writes a claim.
Source resolution	Python resolves those aliases and copies quotation text from the source registry. The model does not rewrite the saved quotations.
Deterministic checks	Citations must resolve to allowed passages; quotations must match normalized source text; numbers in a claim must appear in its evidence.
Support review	A second LLM pass screens claim support. Rejected briefing prose becomes labeled source excerpts; rejected limitations are removed. Copying text does not prove relevance.
Bounded recovery	Schema/provenance failures get one repair attempt, then fail closed. QA has no excerpt fallback: rejected support becomes an evidence-validation abstention.

The honest reliability statement

These layers reduce specific failures; they do not guarantee truth. A real quotation can be misunderstood, and the same local model can miss an error during review. Number matching does not verify units or comparisons. An abstention means the retrieved evidence was insufficient, not that the whole paper contains no answer.

Make the evidence visible

Use the README for installation. Run `papertrail web` for a live local browser interface, or use the CLI commands below. Generation time depends on the machine.

PREPARATION COMMANDS

```
papertrail doctor --warmup
papertrail digest "1706.03762"
papertrail sessions
```

Install the configured local model as described in the README, and start Ollama if needed. Initial model downloads and paper fetching need internet access. Copy the real completed session ID from the output. The arXiv ID above is a reproducible input example; no particular result is promised.

REPLACE SESSION AND CHUNK WITH REAL IDS

```
papertrail show SESSION
papertrail ask SESSION "What problem does this paper address?"
papertrail ask SESSION "What is the main method?"
papertrail ask SESSION "What experiments support the method?"
papertrail inspect SESSION CHUNK
papertrail export SESSION --output exports/demo
```

A compact walkthrough

- **Orient:** show the original input, selected paper and session. Point to the state graph in the README.
- **Audit:** open one accepted claim, read its quotation and follow its page reference into the PDF. Explain what the evidence supports.
- **Question:** ask one specific follow-up. Read the actual result, including an abstention if that is what the system returns.
- **Persist:** exit and reopen the same session. Explain that show/export read saved results, while ask/chat invoke retrieval and generation.

When something fails during a demo

Read the error and saved session ID. For an interrupted pipeline, use `papertrail resume SESSION` after addressing the cause. If QA generation fails on a ready session, retry the ask command. Keep a clearly labeled export of a real saved run available for the walkthrough.

Defend the choices and their cost

The strongest explanation connects a requirement to a design choice, then acknowledges the tradeoff.

CHOICE	REASON	COST / BOUNDARY
Explicit Python graph	Visible state, bounded transitions and simple recovery for a short pipeline	No framework scheduler, distributed execution or complex branching
Qwen + Ollama	Local structured generation without paid API credentials	Hardware-dependent latency; small-model reasoning and context limits
FastEmbed + Qdrant local	CPU embeddings and durable vector search without a separate server	One local writer; not a concurrent multi-user service
SQLite checkpoints	Atomic, inspectable state and persistent question history	Application-level recovery; local artifacts still need to remain intact
Dense + BM25 fusion	Semantic matches plus exact scientific terminology	Additional retrieval logic; usefulness must be measured, not assumed
pdfplumber + page chunks	Transparent extraction with page provenance and modest setup	Heuristic columns/sections; OCR, figures and reliable tables are outside scope
One paper per session	Clear provenance and a bounded, reviewable workflow	No cross-paper synthesis or exhaustive literature search
Local browser UI	Live forms and background jobs reuse the same Python agent	Loopback only; job status is in memory. Public Pages serves saved results.

Where reliability can still break

Search can select a weak candidate. Parsing can lose an equation or table relationship. Retrieval can miss the decisive passage. Generation can overstate a supported fact. The review model can accept the same mistake. Inspect the failure stage before changing prompts or choosing a larger model.

A credible next step

Build a larger, held-out question set across papers and layouts. Label the passages needed to answer each question, compare dense/BM25/hybrid retrieval, then review answer support and abstention quality. Improve the component responsible for measured errors before adding more infrastructure.

Answers you can explain in your words

Practise the reasoning, then point to code or a real artifact. Do not claim results you have not verified.

Why is this a graph instead of one prompt?

Each stage has a clear input, output and failure point. The saved next-node field tells the application where to resume. The LLM does not choose arbitrary tools or control execution.

Why do you need a vector database and SQLite?

They hold different data. Qdrant retrieves passage vectors and payloads. SQLite stores application state, selected metadata and QA history. The original PDF and parsed text remain separate files.

How is RAG different from fine-tuning?

RAG supplies relevant paper text when a question is asked. No model weights are changed here. Fine-tuning would change weights using training examples and would not remove the need for source evidence.

Does a citation mean an answer is correct?

It proves a narrower point: the cited text is traceable to the source. Code checks the provenance, and a second model pass screens the meaning. A person still needs to assess whether the claim faithfully follows from the quotation.

How do you handle follow-up questions?

Questions and answers persist in the session. A simple referential-query rule can append the previous question to retrieval. The model sees recent questions as context, but past generated answers cannot become factual evidence.

How would you evaluate this fairly?

Separate retrieval from answer quality. Check whether the required source passages are retrieved, then check claim support and appropriate abstention. Use held-out questions and publish actual results, including cases where hybrid retrieval loses.

What would you change for production?

First establish an evaluation baseline. Then separate jobs from the interface, use concurrent-safe storage, add access controls and resource limits, and monitor retrieval and validation failures. The current application is deliberately local and single-user.

Code reading route: schema.py -> graph.py -> sources.py / parsing.py -> retrieval.py -> llm.py / grounding.py -> storage.py -> cli.py / rendering.py.

A four-minute story with evidence

This is a preparation outline. Adapt the first-person draft in docs/reflection-script.md to match your actual contribution and experience.

TARGET	WHAT TO COVER	WHAT TO SHOW
0:00-0:30	Problem, intended reader and one-paper scope	Repository title and a saved briefing
0:30-1:10	Finite graph, typed state and recovery	README state graph and a session record
1:10-2:00	Retrieval, evidence aliases, code-owned quotes and support review	One claim beside its source quotation
2:00-2:45	A follow-up and durable session history	Actual CLI output or a clearly labeled saved run
2:45-3:30	Tradeoff, limitation and measured improvement plan	One real issue you can explain
3:30-3:50	Your own contribution and what you learned	Concise closing statement in your own words

The target leaves ten seconds of buffer. Rehearse with a timer; these are presentation allocations, not observed application latency. Show a real result rather than promising a particular answer.

Before you submit

Confirm the public repository opens, setup instructions match a fresh environment, example QA comes from a saved run, and any reported checks link to actual evidence. Record the reflection yourself. Keep secrets, downloaded PDFs, local databases and model weights out of the repository.

Where to read next

PaperTrail source, README and technical notes
<https://github.com/Gaganpraveen/papertrail>

arXiv API user manual - metadata search interface
<https://info.arxiv.org/help/api/user-manual.html>

Ollama structured outputs - JSON schema responses
<https://docs.ollama.com/capabilities/structured-outputs>

Qdrant Python client - local mode
<https://github.com/qdrant/qdrant-client#local-mode>

Implementation descriptions were checked against the local source. External links are official reference material; they do not certify this application's behavior. This guide intentionally makes no benchmark, test-count or demo-duration claims.